RetryVLA

World-model-guided synthetic recovery learning for VLAs

Progress M0–M5 · LIBERO-Recover Spatial · Oct 2026

The question we are answering

Can a world model discover action sequences that recover from actual VLA failure states, and can those imagined recoveries be distilled into the VLA so it recovers without search at inference?

1

Failure states

LIBERO-Recover scenes where SmolVLA failed

2

Search

propose 50-step recovery segments, score them

3

Verify

run best candidates in the simulator + SmolVLA continuation

4

Distill

fine-tune SmolVLA on verified recoveries

5

Evaluate

recovery success on held-out failures, no search

Search is where the world model enters: it replaces the simulator as the thing that says which candidates are good.

Setting: benchmark, policy, compute

Benchmark

- LIBERO-Recover, Spatial suite
- 168 failure scenes, 84 failed episodes, 9 tasks
- official protocol: 5 closed-gripper + 5 open-gripper trials, ±2 cm object noise, 520 steps
- metric: Recovery Success Rate (RSR)

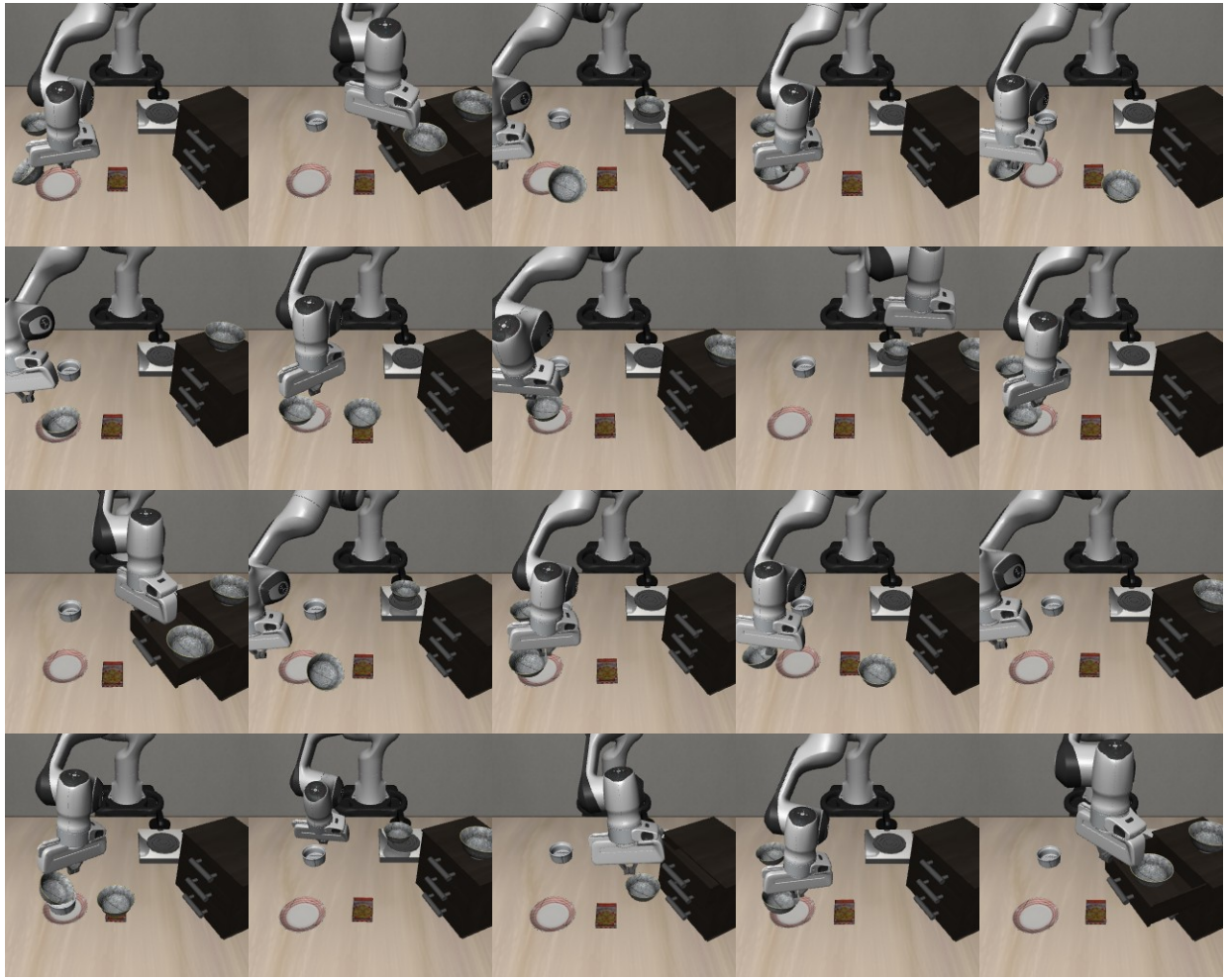
Policy

- SmolVLA, checkpoint HuggingFaceVLA/smolvla_libero (450M)
- chunk 50 actions, replan every 10
- 7-D OSC delta actions, gripper ±1
- pi0 = checkpoint as-is; pi1 = after recovery fine-tuning

Compute

- 1 local A100 + idle A100s borrowed from other shells (avoid-list)
- MuJoCo rendered on GPU (user-local NVIDIA EGL), ~20 ms/step
- bit-exact sim snapshots for branching
- all runs tracked: config, git sha, metrics, registry

What a LIBERO-Recover failure state looks like

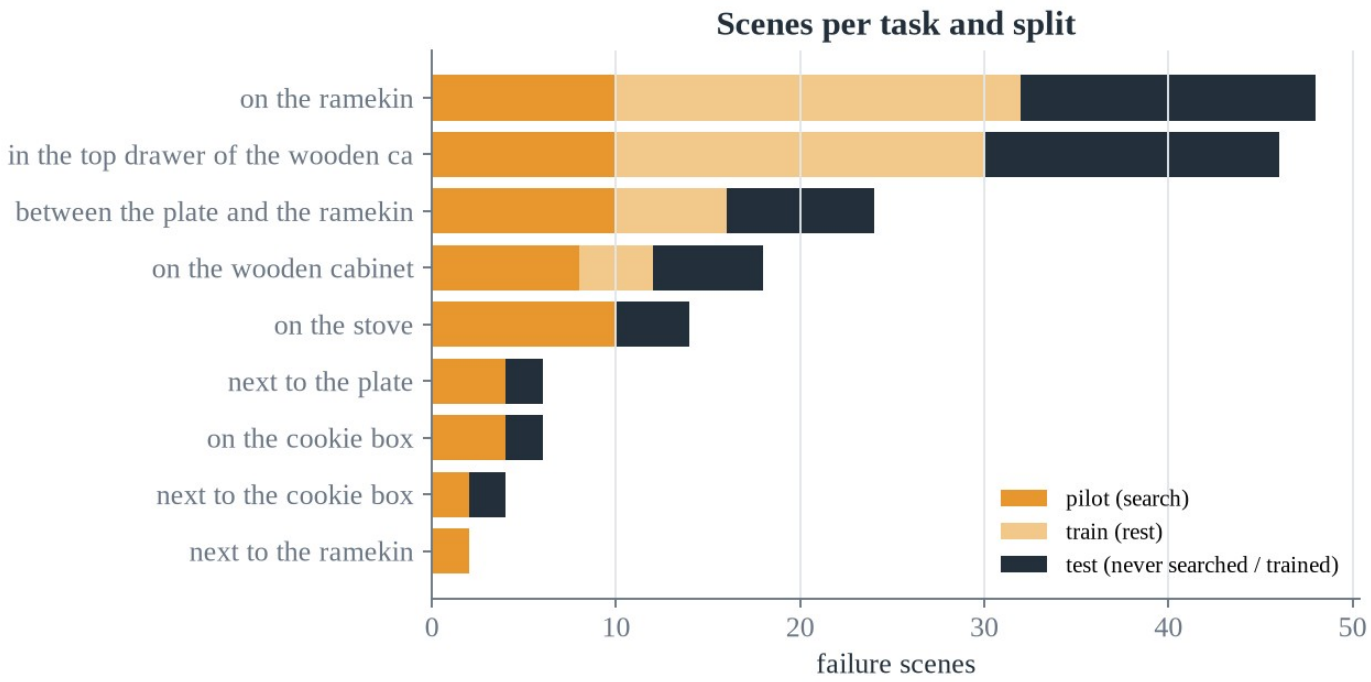


20 restored failure states (agent camera), M0b check: 20/20 deterministic

Each scene = a .bddl task + the exact physics state recorded when SmolVLA was failing

- bowl half-grasped, pushed off the plate, stuck in a drawer, arm in an odd pose
- the gripper is forced closed in half of the trials: an initial state SmolVLA never saw in training
- restore is bit-exact: state, controller, observables. The same actions always give the same pixels
- no L1-L4 difficulty labels released yet

Train / pilot / test split, grouped by failure episode



112

train scenes (search, WM data, SFT)

60

pilot scenes (subset of train, search targets)

56

test scenes: never searched, never trained on

step_0180 and step_0200 of one failed episode are near-duplicates, so they always stay in the same split. Code asserts test scenes never enter search or training.

M0: SmolVLA is decent at nominal tasks, bad at recovery

0.66

nominal LIBERO-Spatial success (500 ep)
lerobot-eval cross-check: 0.68

0.12

recovery success (RSR) on 56 held-out failure scenes, 560 ep

5 / 160

drawer-task failures recovered



Training data SmolVLA learned from: a nominal LIBERO-Spatial demo (agent + wrist camera, action per step)

- checkpoint is weaker than the ~0.90 reported for SmolVLA, but our engine matches lerobot-eval, so the interface is correct
- fine-tuning on the Spatial demos alone reaches 0.80 nominal. That checkpoint (pi1-nominal) becomes the control and the stronger base
- replanning every step (n_action_steps=1) gives +5 points at 10x the cost, so we keep 10

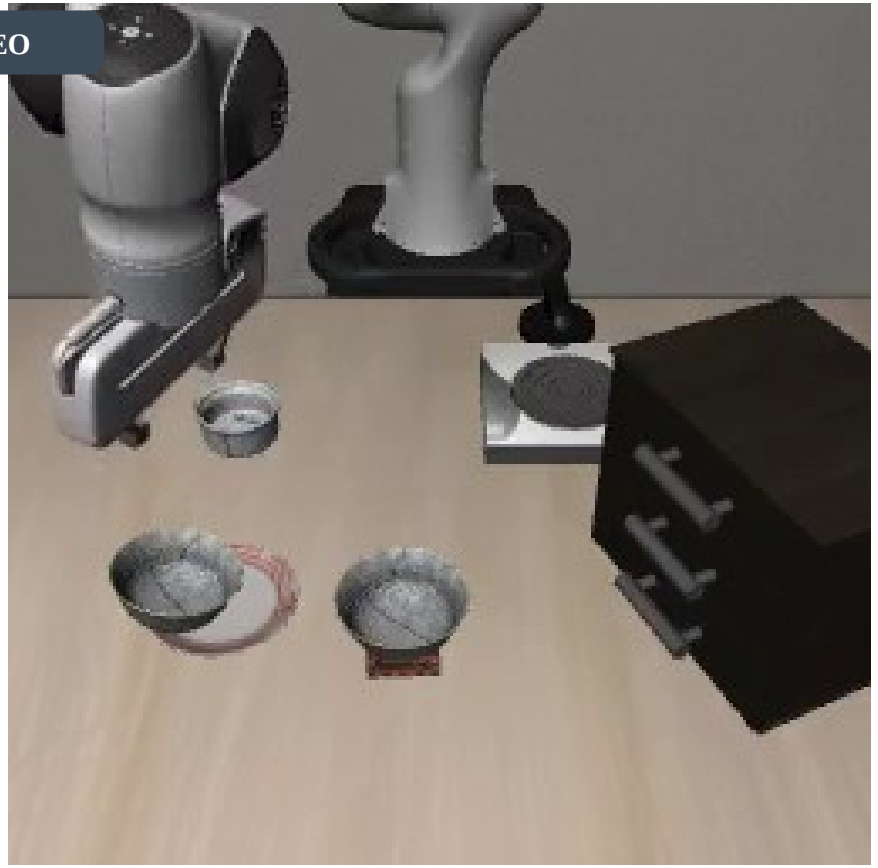
Video: SmolVLA on held-out failure states

VIDEO



Drawer failure: pi0 never recovers (26 s)

VIDEO



Ramekin failure: pi0 keeps trying the same grasp (26 s)

Official protocol, 520 steps. These are the 88% of test episodes we want to fix.

M1: how one recovery candidate is tried

Restore

load the exact failure snapshot

Segment (50 steps)

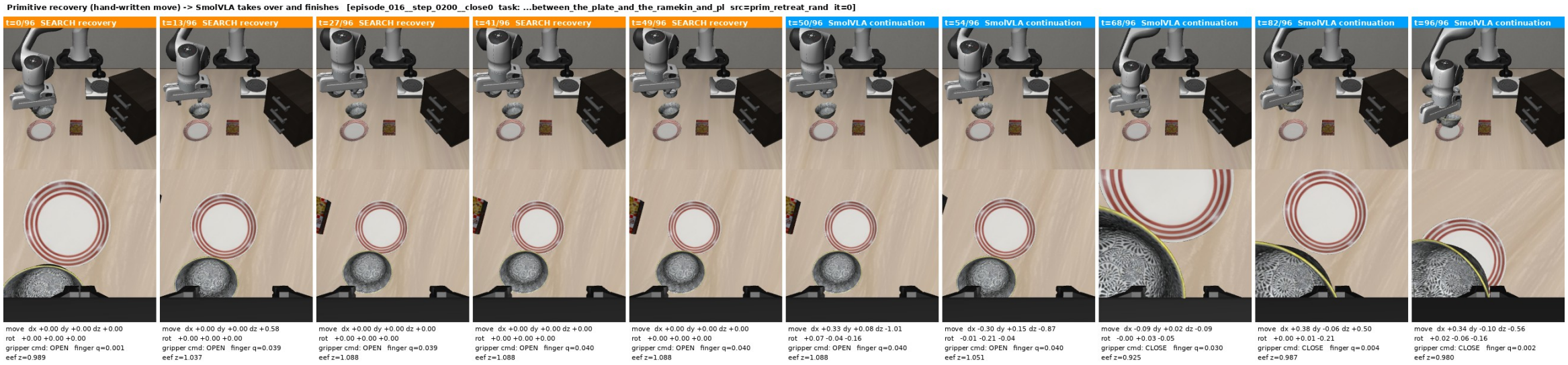
open-loop actions from search: SmolVLA samples, smooth perturbations, primitives

SmolVLA continues

frozen pi0 drives closed-loop up to 300 steps

Judge

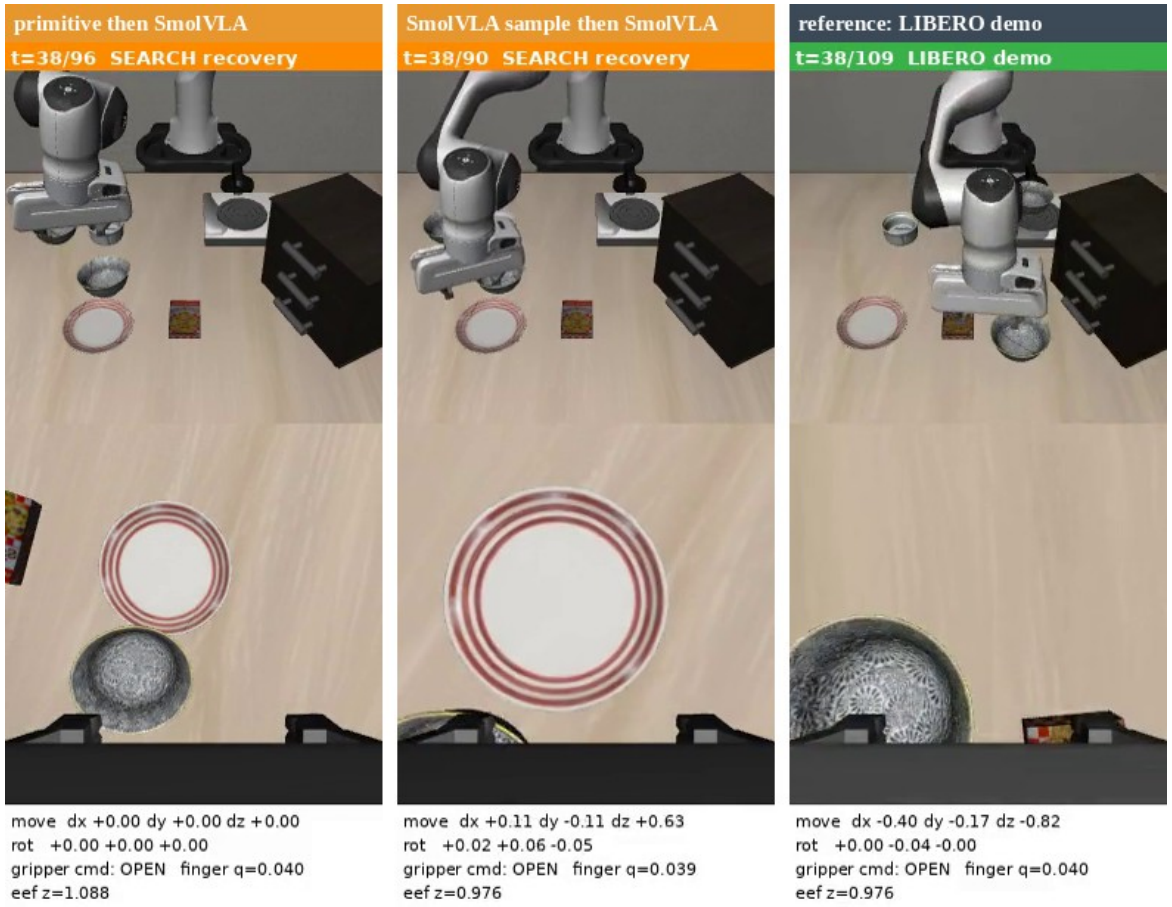
simulator checks the BDDL goal: bowl on plate?



Orange = searched segment (here a hand-written 'open + lift' primitive). Blue = SmolVLA takes over and places the bowl. Competence-targeted: the segment does not solve the task, it returns the robot to a state SmolVLA can finish from.

Video: oracle-search recoveries

VIDEO



Orange bar = searched 50-step segment, blue = SmolVLA continuing, green = nominal demo. Overlay: step, segment, action given to the policy, finger opening.

M1: search = propose, score in the simulator, refine

1

Propose 64

24 SmolVLA samples (different noise), 24 smooth perturbations, 16 primitives

2

Score

simulate each: success × speed + small goal-progress shaping − jerk

3

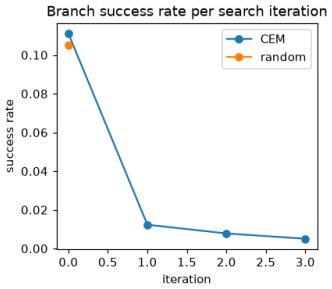
Keep elites

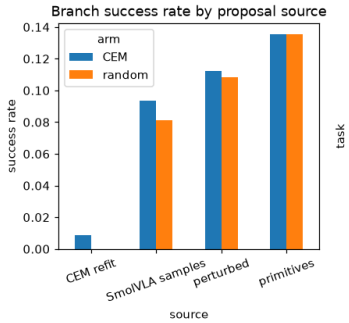
8 best candidates

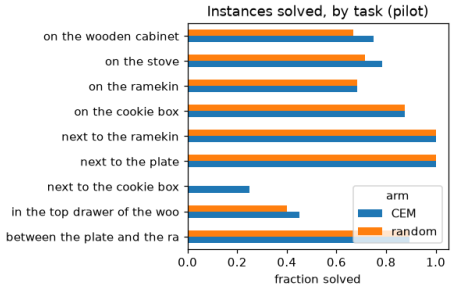
4

Refine

CEM: sample around elites, up to 4 rounds (≤256 sims / instance)



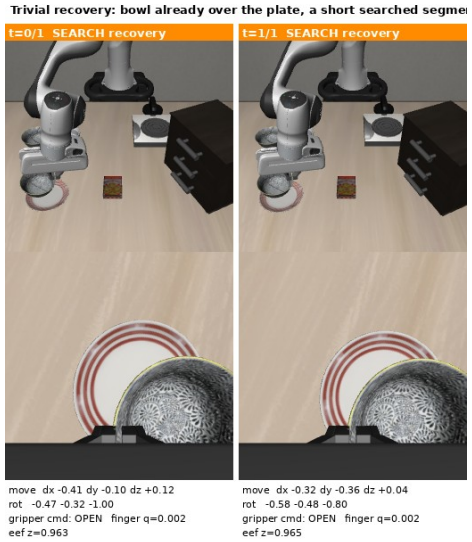




What we learned

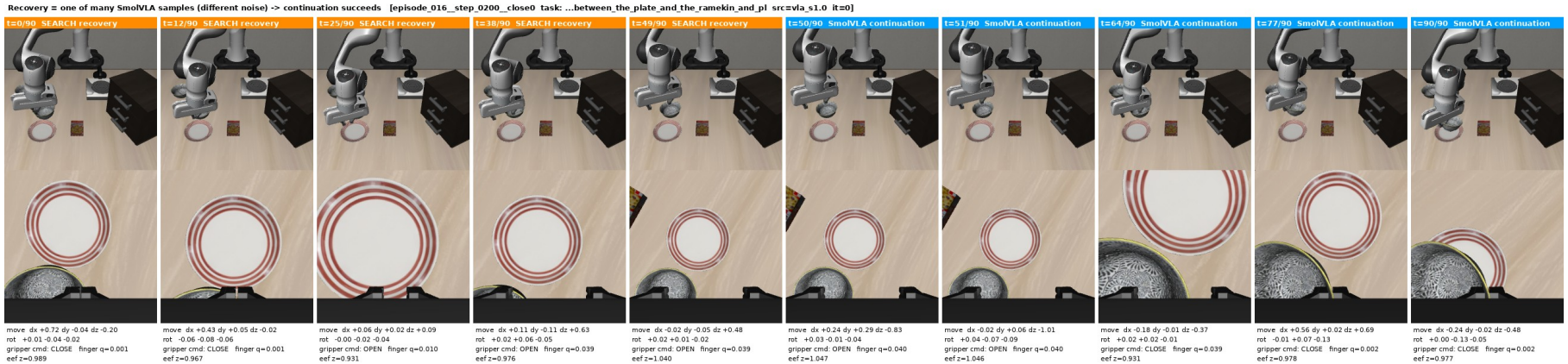
- ~70% of pilot failures have a recovery reachable in 50 steps
- the first CEM was broken: averaging different strategies into one Gaussian drops success 11% → 1%
- fix: refine around ONE elite at a time. On hard instances it then solves 68% vs 55% for random at equal budget
- ~1/3 of recoveries are trivial (bowl already over plate: just open)

What the recovery training data looks like

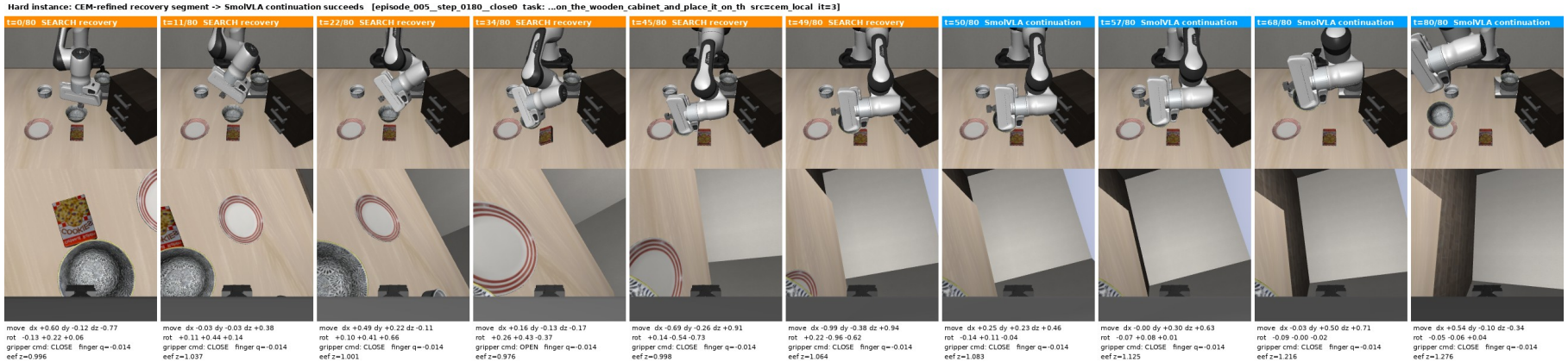


Trivial

2 steps: release the bowl already above the plate. Removing these HURT RSR (0.254 → 0.220)



One of 24 SmolVLA samples (different noise) happens to open and back off; continuation succeeds



Old CEM on a hard instance: jerky, gripper chatter (3-4 flips / 50 steps vs 0.9 in demos). Searched segments are 4-6x jerkier than demos

Video: why random-noise search proves little

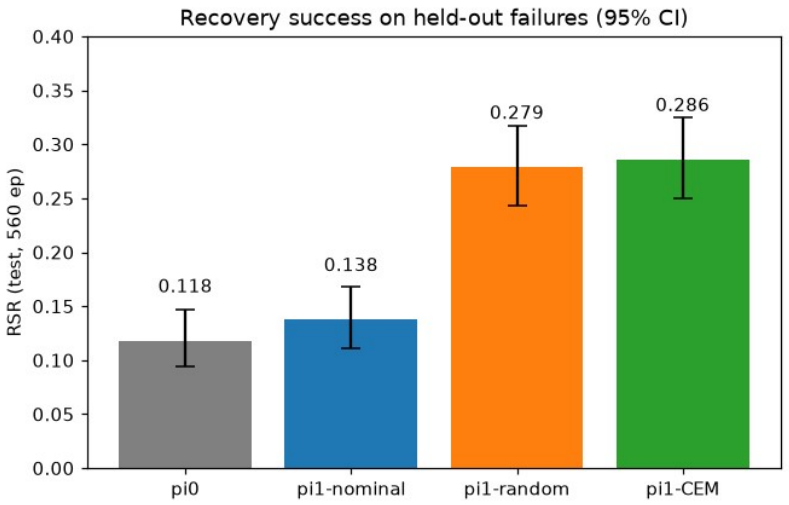
VIDEO

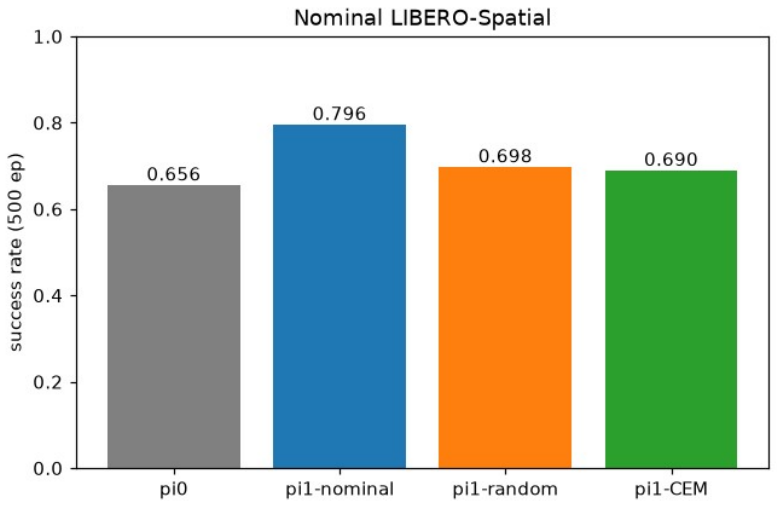


Old CEM on a hard instance

- arm thrashes, gripper flips open/closed with no effect on the fingers
- it still 'succeeds' afterwards, so it lands in the training set
- 4-6× jerkier than demos: a likely reason recovery SFT costs nominal success
- motivates: smoothness cost, elite-local refinement, and scoring by a model of what SmolVLA can finish

Distilling simulator-verified recoveries works





SFT arm	recovery data	test RSR	nominal
pi0	-	0.118	0.656
pi1-nominal (control)	demos only	0.138	0.796
pi1-random	241 random-search	0.279	0.698
pi1-CEM	241 CEM	0.286	0.690
ratio 0.25	241 random	0.254	0.726
ratio 0.25, no trivial	170 random	0.220	0.766

2.4×

test RSR vs pi0 (0.118 → 0.28), +14 pts over the fine-tune-only control

- CEM ≈ random here: the old CEM's useful data all came from its random first round
- cost: nominal 0.80 → 0.70 at a 50% mix. Trade-off set by the ratio
- trivial recoveries are useful supervision

Replacing the simulator with a world model

M2: latent world model + competence critic · M3: search in imagination

Why a world model if we already have a simulator?

Oracle search (M1)

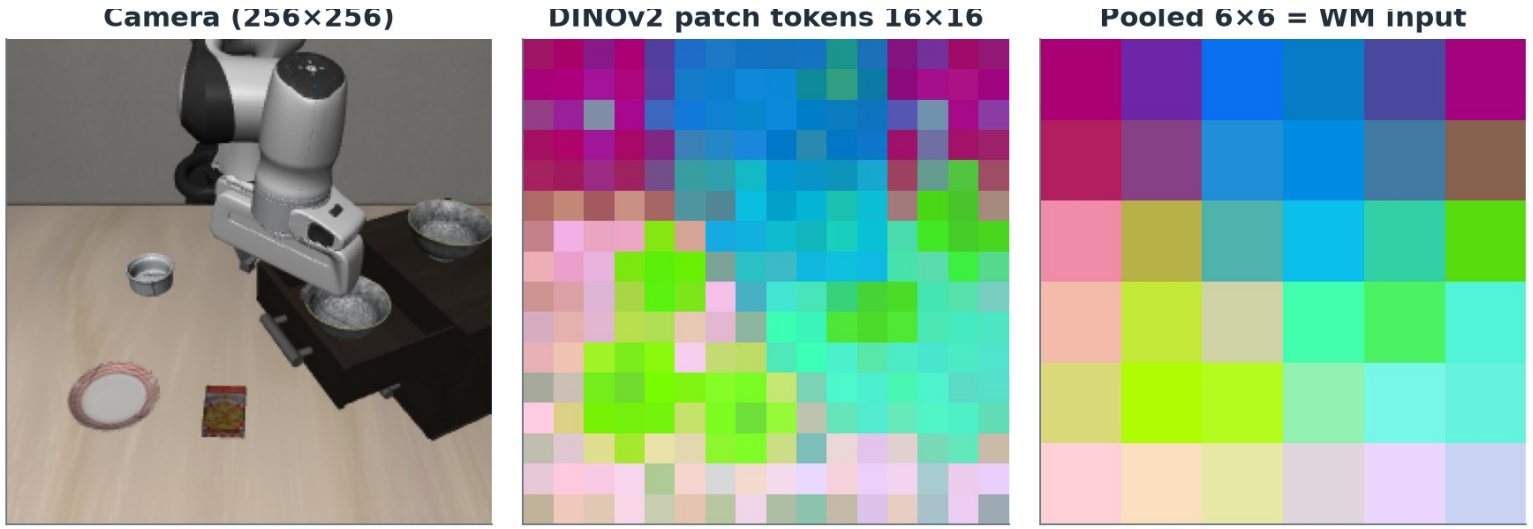
- scores a candidate by running it for real
- up to 256 simulator rollouts per failure
- exact, but only exists where a simulator of THIS scene exists
- upper bound on what search can find

World-model search (M3)

- scores a candidate by imagining it in latent space
- ~800 candidates in a few GPU seconds
- simulator only to verify the top 16 (later: real robot, or none)
- needs: accurate dynamics on failure states + a critic that knows what SmolVLA can finish

Claim to test: at a fixed number of real rollouts, WM-guided search finds more recoveries than proposing at random.

State = frozen DINOv2 patch tokens + robot state



Same drawer failure state. Token colors = first 3 PCA components: bowl, drawer, arm and table separate without any training

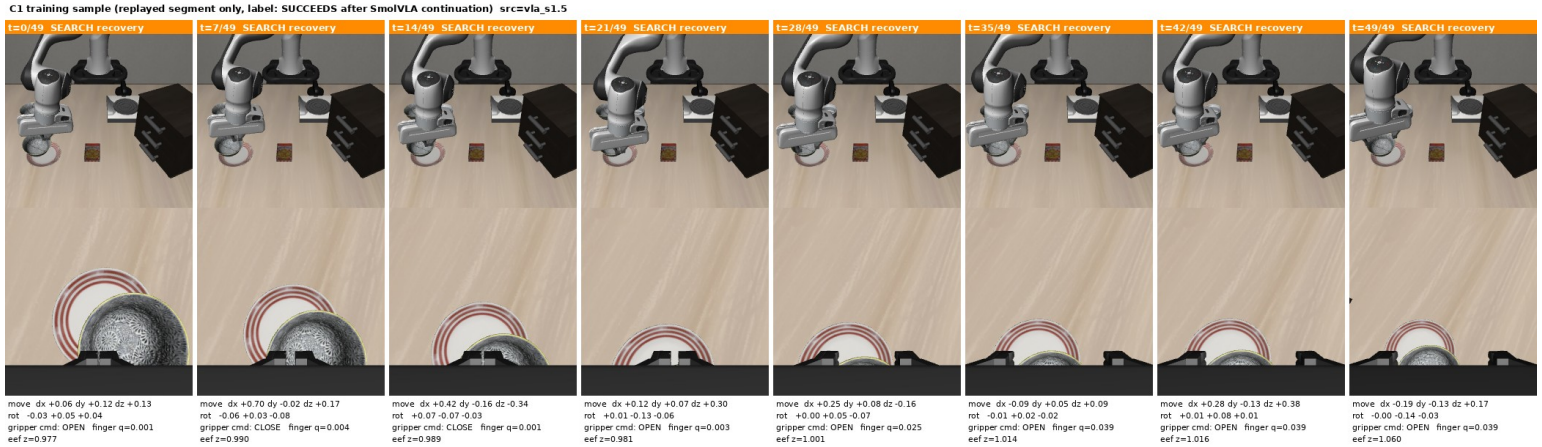
C1 training data for the WM and critic

- 144 TRAIN failure instances × 32 candidate segments = 4,608 simulated trajectories
- latents every 5 env steps + the actions + outcome label (did SmolVLA finish afterwards?)
- the failure episodes containing the 53 hard search targets are excluded: those failures stay UNSEEN

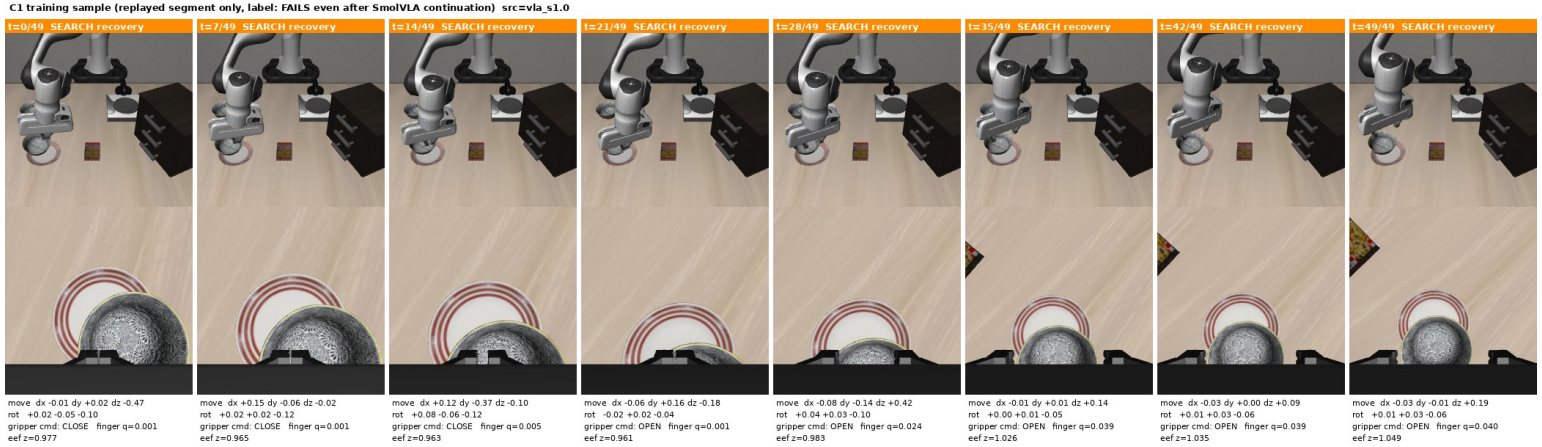
h_t (per time step)

- agent cam 6×6 tokens
- wrist cam 6×6 tokens
- robot state q (8-D): eef pos, axis-angle, finger
- = 72 tokens × 384-D + 1 state token
- spatial tokens, not one global vector: L3/L4 failures depend on where things are

One failure state, two candidates, two labels

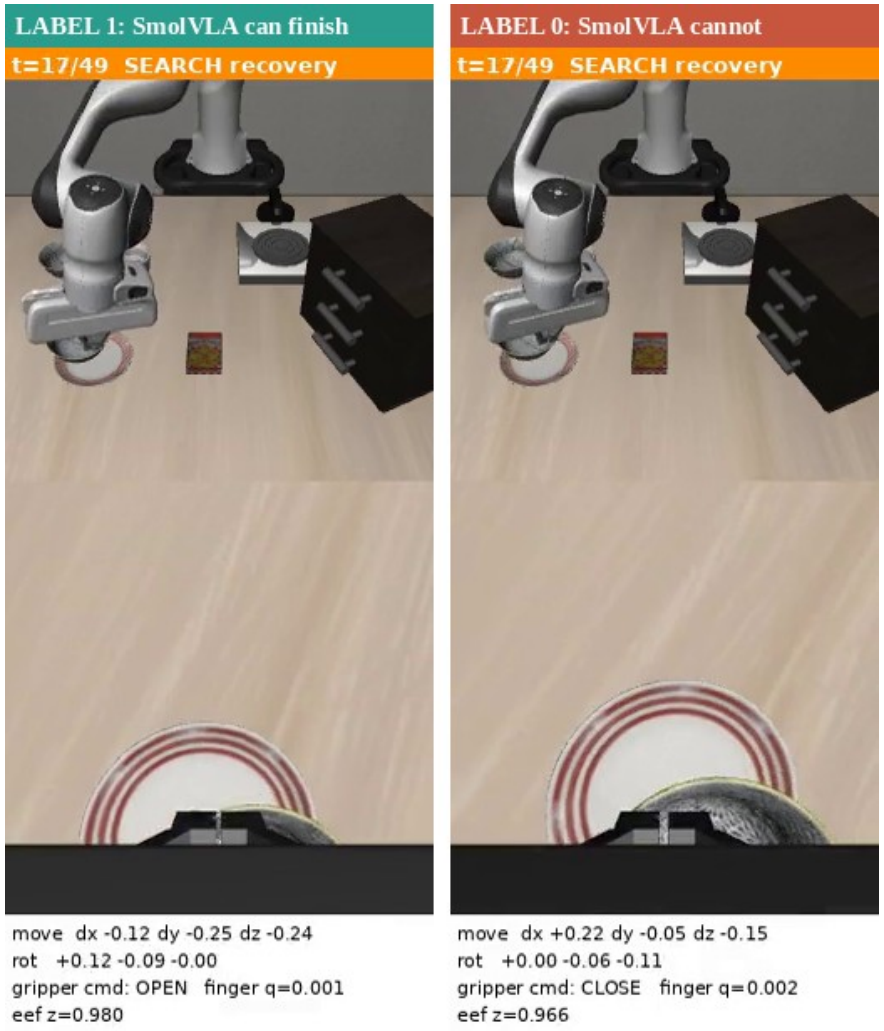


Label 1: opens at t=21 and lifts. The bowl stays on the plate and SmolVLA finishes



Label 0: also opens, but drags the bowl off the plate (t≥35), so SmolVLA cannot finish. The critic must tell these apart from latents alone

Video: WM data, same state, opposite labels



Same failure state, two candidate segments

- left: opens and lifts, bowl stays on the plate, so SmolVLA can finish (label 1)
- right: opens but drags the bowl off the plate, so SmolVLA cannot (label 0)
- 4,608 such segments teach the WM how latents move and the critic which end states are recoverable
- replayed from the stored actions: bit-exact

M2: latent dynamics h(t+1) = h(t) + F(h(t), 5 actions)

h_t

72 patch + 1 state token

+ action token

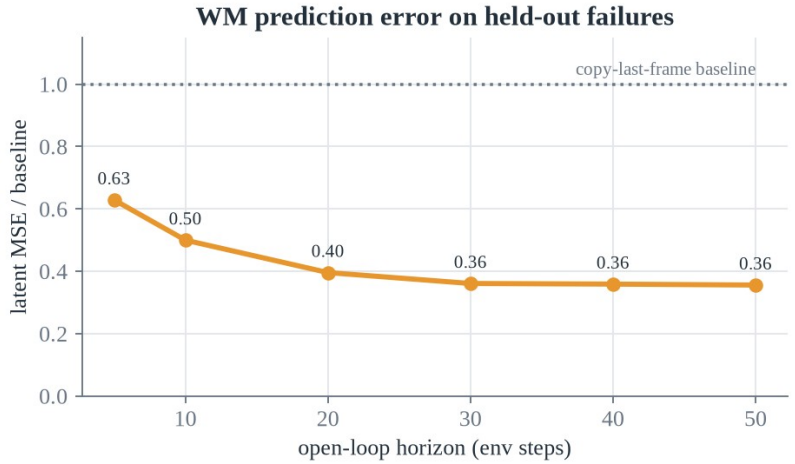
MLP(5 × 7-D actions)

Transformer

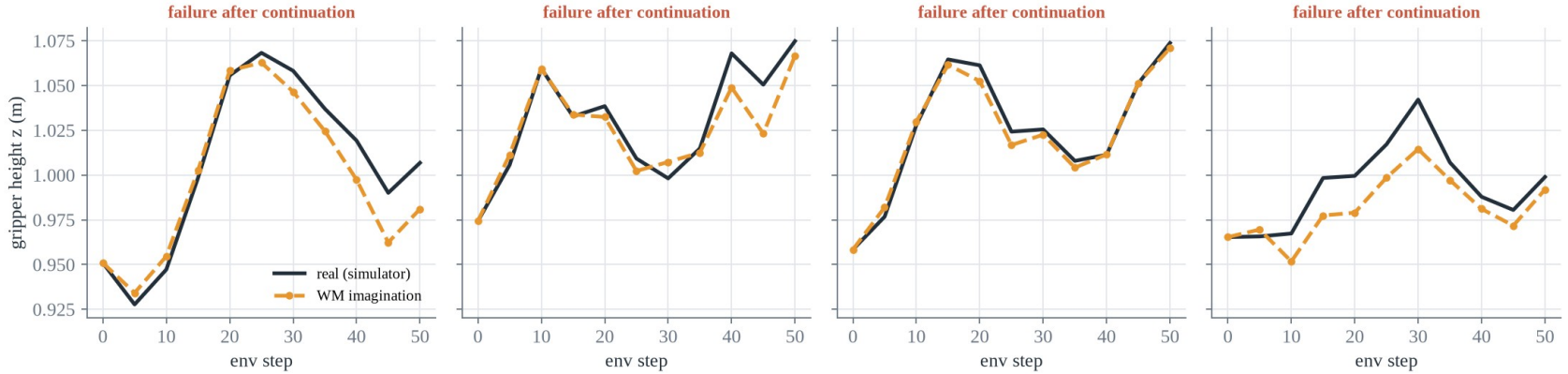
6 layers, width 384

4 heads

ensemble Δh: mean = prediction, spread = uncertainty



- loss = 1-step teacher-forced + 10-step open-loop rollout (search needs 50-step imagination)
- 11.6M params, 20k steps, ~1.5 h on one A100

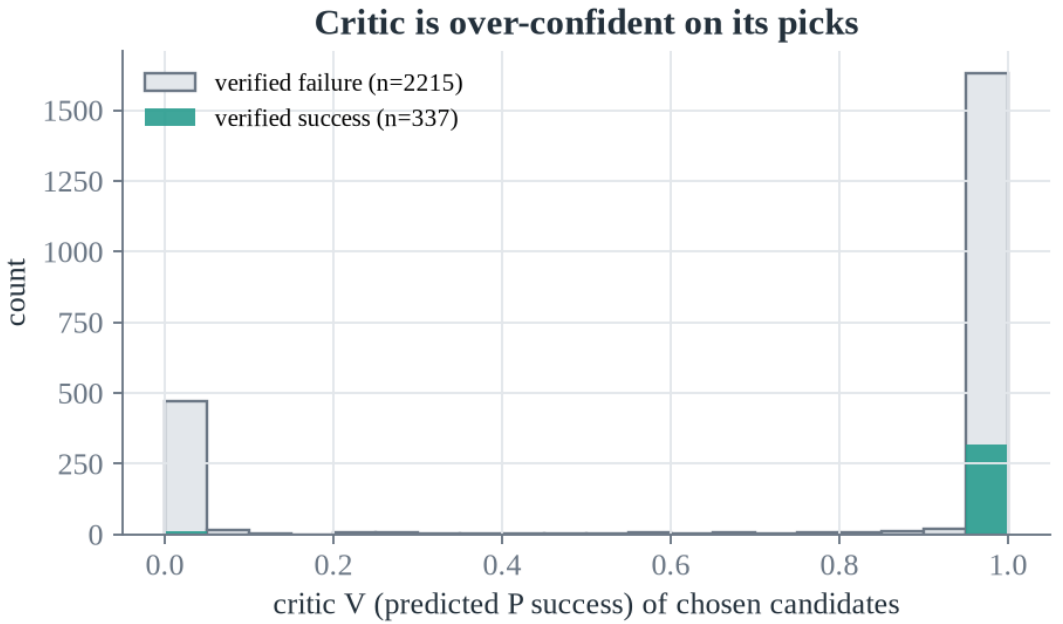


Held-out failures: gripper height, real (black) vs imagined from the start frame + actions only (amber). Error is 0.36-0.63× the copy-last-frame baseline

M2: the competence critic V(z, q, task)

“From this (possibly imagined) state, will frozen SmolVLA finish the task?”

- small transformer over the same tokens + learned task embedding
- trained on real AND WM-imagined end states (50/50), so it is calibrated for what search feeds it
- competence-targeted: lifting out of a drawer scores high even though the bowl gets no closer to the plate
- held-out AUC 0.71 real / 0.67 imagined; top-4 picks succeed 2.1× the base rate



Ranks well, calibrated badly

Almost every pick gets V≈1 while ~12% verify. Fine for ranking. Needs fixing before V can weight training (AWR).

M3: how the world model searches

1 Encode

real failure frame → DINOv2 latent z0, robot state q0

2 Propose 256

same proposal pool as M1: SmolVLA samples, perturbations, primitives

3 Imagine

WM rolls each 50-step candidate forward: z0 → z50 (10 steps of 5 actions)

4 Score

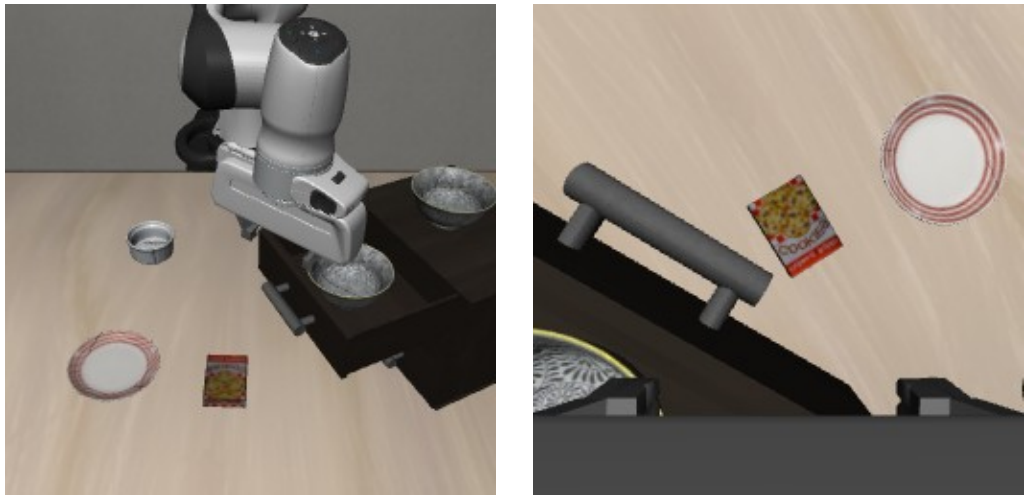
$S = V(z50) - \lambda \cdot \text{uncertainty} - \beta \cdot \text{jerkiness}$

5 Refine in imagination

2 rounds of elite-local CEM, ~800 candidates, zero simulator calls

6 Verify top-16

only these touch the simulator; successes become training data

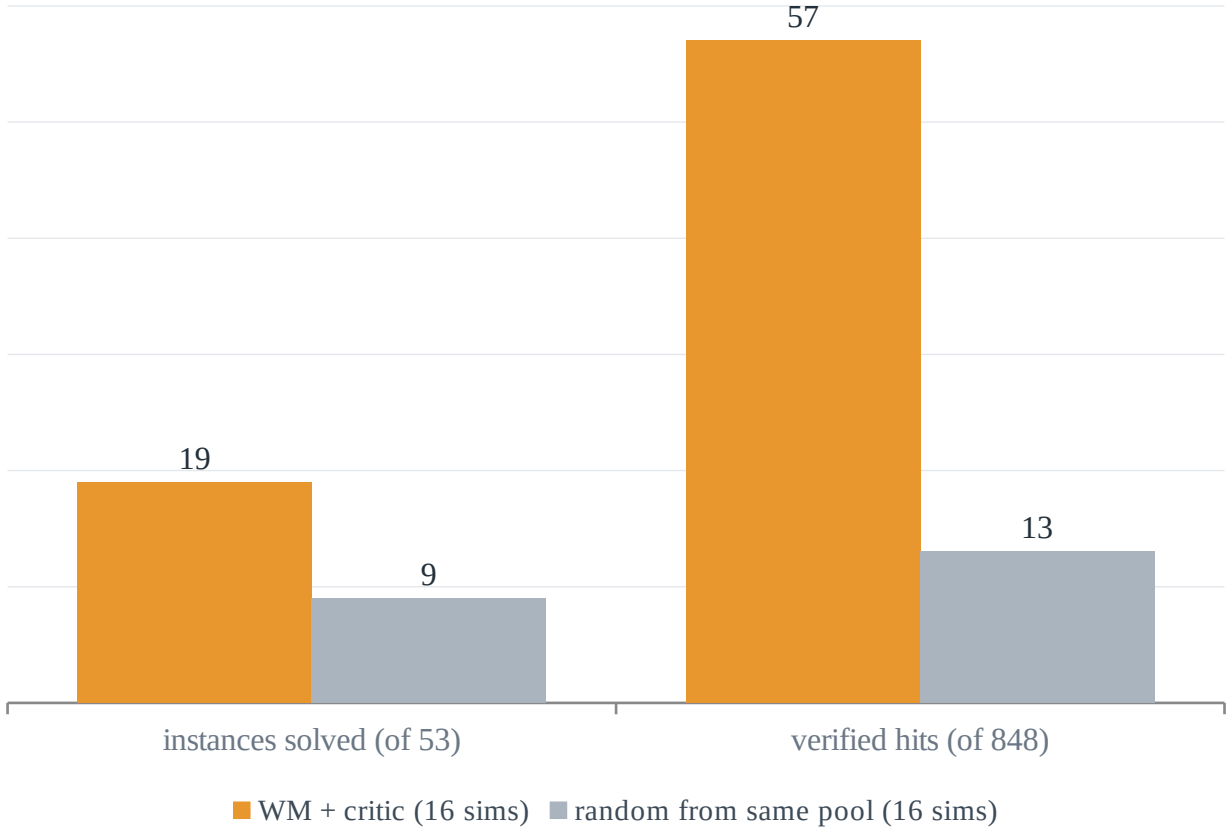


Input: one real failure observation (agent + wrist)

Fair comparison

- random arm: 16 candidates drawn from the same 256-proposal pool
- both arms: exactly 16 simulator rollouts per instance
- 53 hard instances whose failure episodes the WM/critic never saw

M3: 2× more recoveries on unseen hard failures



Picks are also smoother: jerk 0.017 vs 0.027

36% vs 17%

hard failures solved, same simulator budget (sign test p = 0.04)

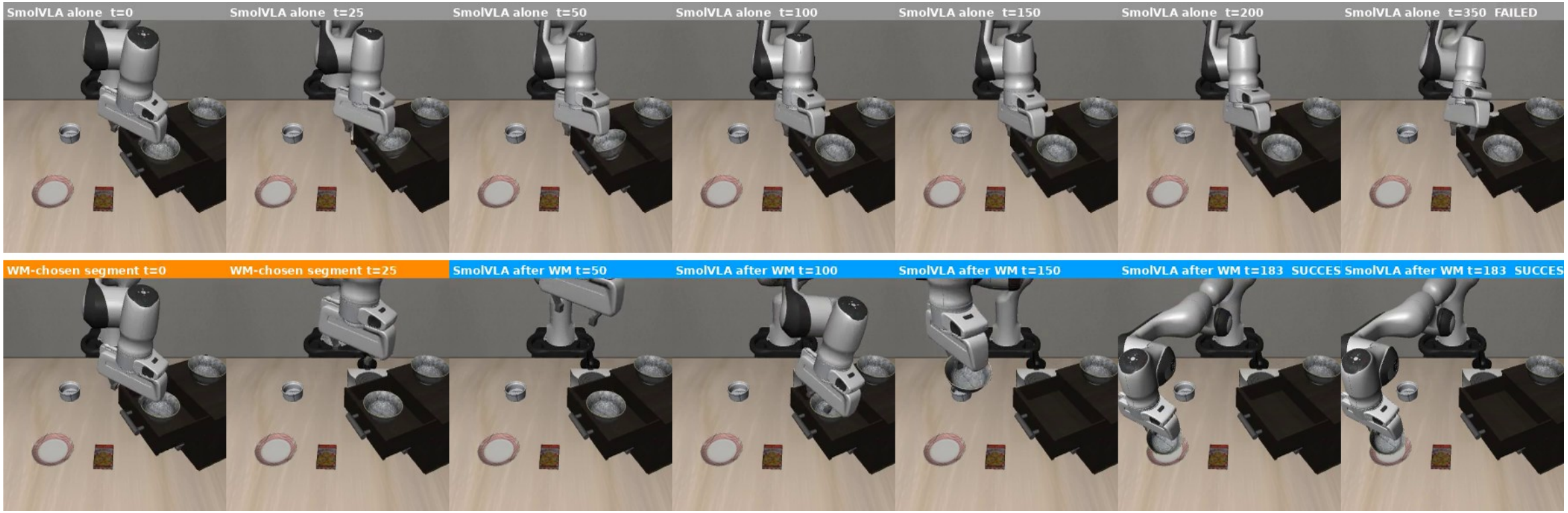
35 / 45

recoveries found by latent CEM: plans the WM invented, never simulated before verification

Caveat

The gain is concentrated in the drawer task (WM 12/20 vs random 1/20). Other tasks: 7/33 vs 8/33. N = 53, one seed.

Same failure state: SmolVLA alone vs WM-guided



Top: SmolVLA alone hovers at the drawer mouth for 350 steps. 0/3 seeds, 0/4 pre-checks, random 0/16.

Bottom: the WM picks 'lift straight up' (orange). From there SmolVLA reaches in, grasps and places the bowl. 3/16 verified.

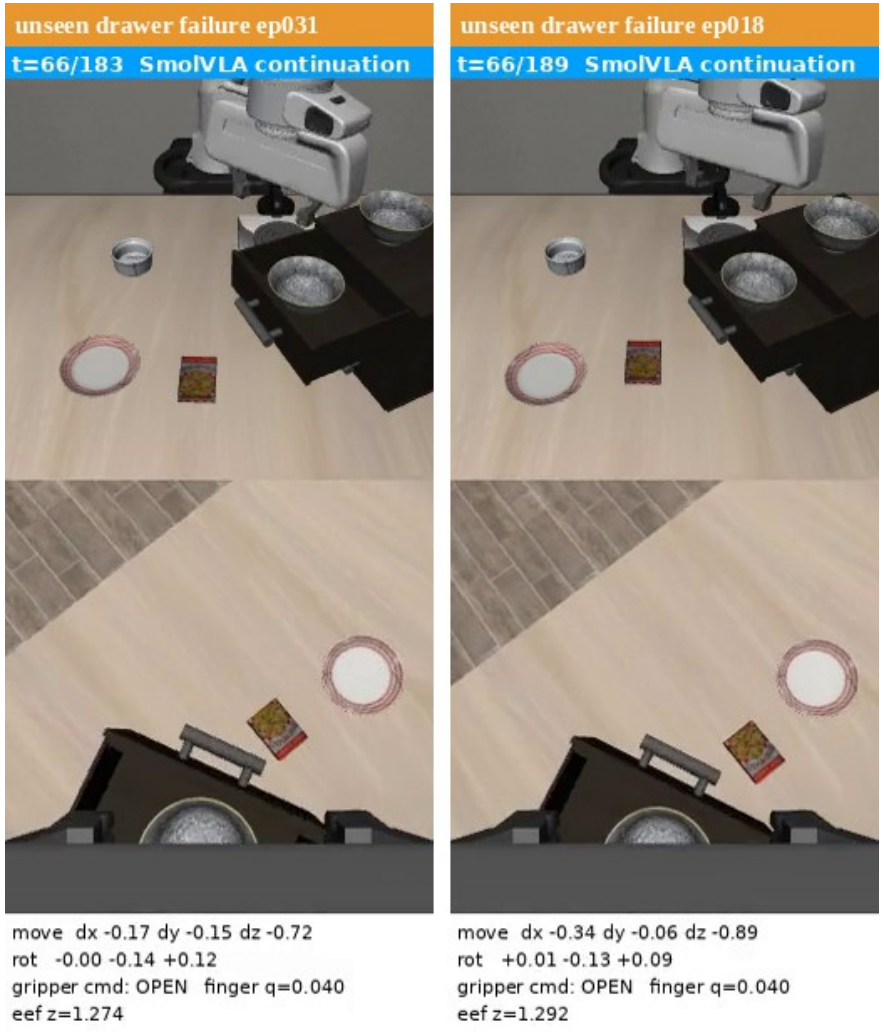
Video: SmolVLA alone vs WM-guided, same failure

VIDEO



Left: SmolVLA alone hovers at the drawer for 350 steps. Right: WM-chosen 50-step lift (orange) then SmolVLA grasps and places the bowl.

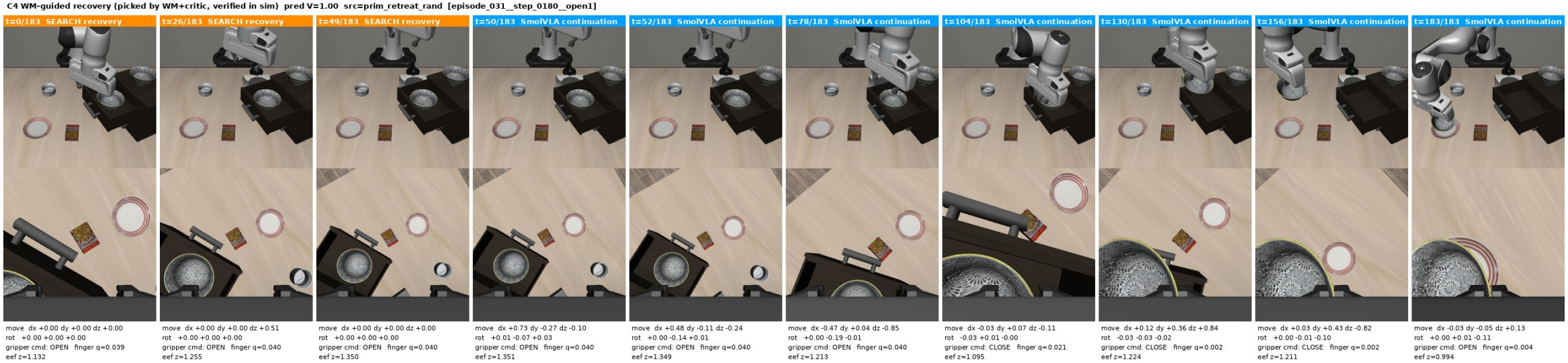
Video: WM-found recoveries on unseen failures

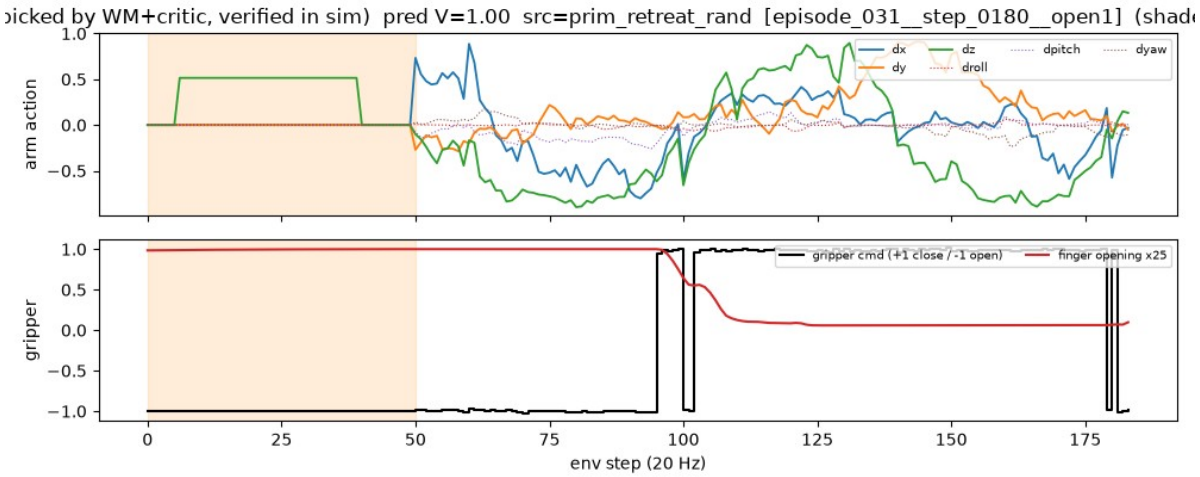


Picked by WM + critic, verified in sim

- orange: the 50 steps the WM chose in imagination (lift clear of the drawer)
- blue: SmolVLA reaches in, grasps, places the bowl
- neither failure episode was in the WM's training data
- same strategy found on different failures: the WM generalizes the 'get clear first' idea

What the WM chose, step by step





- shaded = the 50 steps the WM chose: one clean dz = +0.5 move, gripper open, no chatter
- white = SmolVLA's own actions afterwards; finger closes at t≈95 (red), bowl placed at t=183
- contrast with the jerky old-CEM segment in M1

M4: does WM-found data improve the VLA? Not yet measurable

policy (init)	recovery data	test RSR	drawer	nominal
pi1-random (pi0)	241 random	0.279	18/160	0.698
A (pi1-nominal)	241 random	0.293	24/160	0.756
B (pi1-nominal)	241 random + 45 WM	0.298	28/160	0.748

- stronger init (pi1-nominal) recovers nominal 0.70 → 0.75 and nudges RSR up
- +45 WM trajectories: +0.005 RSR, drawer 24 → 28. Paired episodes 61 vs 58: noise
- the dose is tiny: 45 WM trajectories ≈ 8% of training samples

So M5 asks it properly

- generate WM data on all 108 pilot failures (not just 53 hard)
- same trajectory count WM vs random
- WM-only vs random-only SFT, same init, same recipe
- measure RSR, drawer, nominal

M5 (running now): more, more diverse, cheaper WM data

1

Diverse top-k

k-means over imagined end states, best candidate per cluster, so the budget is not spent on 16 copies of one plan

2

Adaptive budget

verify in batches of 8; stop at 4 successes; ≤32 sims. Easy failures cost 8, hard ones get more

3

Self-improving critic

every verification (success AND failure) is saved as a label exactly where the critic was wrong → critic v2 → re-run on 50 held-out instances

4

Matched distillation

SFT WM-only vs random-only, same #trajectories, init pi1-nominal → test RSR + nominal

Round 1 so far (82 / 108 instances)

~76%

of pilot failures get ≥1 verified recovery

~13%

verified yield per simulator rollout (M3: 6.7%; random: 1.5%)

216

WM trajectories written so far (M3 had 45)

Interim numbers from the live run on 4 borrowed A100s; final results tonight

Where we stand

Supported so far

- recoveries exist and can be found: ~70% of failures, 50-step segments
- distilling them works: test RSR 0.12 → 0.29, beyond a fine-tune-only control
- a latent WM + competence critic finds recoveries on unseen hard failures 2× more often than random at equal simulator budget
- most of those were invented by the WM in imagination

Open problems

- WM data → better VLA is not shown yet (M5 tests it)
- WM gain mostly on one task (drawer); critic over-confident
- recovery SFT costs nominal success (0.80 → 0.75)
- weak base checkpoint (0.66 vs ~0.90 reported)
- one suite, one seed, 56 test scenes
- searched segments jerkier than demos

Long-term plan

M5	**Now**	prove WM data > random data for the VLA at equal count; self-improving critic
M6	**Next**	closed loop π0 → π1 → π2: mine pi1's remaining train failures, search, retrain. Prioritize failures the WM solves but the policy does not
M7	**Then**	quality-diversity search (MAP-Elites over recovery modes), parameterized primitives, multi-chunk / MCTS for multi-stage recoveries
M8	**Scale**	Object, Goal, LIBERO-10 suites; L1-L4 levels once released; 3 seeds; stronger base VLAs
V2	**Stretch**	neural video WM generates RGB for imagined recoveries (no simulator at all), then real-robot recovery

The paper story we are building toward

Competence-targeted world-model search generates recovery actions that restore the policy's ability to finish the task, and distilling them lets the VLA recover from unseen failures without search.

1 **Goal-progress vs competence critic**

does scoring by 'can SmolVLA finish' beat 'is the bowl closer'?

2 **Random vs oracle vs WM selection**

same #trajectories: is the gain from the WM, not just more data?

3 **Sim-efficiency curve**

verified recoveries per simulator rollout: random, oracle CEM, WM

4 **Iterations π0 → π1 → π2**

does the loop keep improving RSR on held-out failures?

Next decisions

Tonight: M5 results (WM-only vs random-only SFT, critic v1 vs v2)

If WM data wins → loop π1 → π2 and diversity search. If not → fix data quality (smoothing, gripper latch) before scaling